

Informe de Arquitectura del Sistema

1. Introducción

El presente documento describe la arquitectura utilizada para el desarrollo del sistema, la cual se basa en principios de Clean Architecture y arquitectura por capas, junto con la implementación de diversos patrones de diseño y buenas prácticas de desarrollo.

El objetivo de esta arquitectura es lograr un sistema modular, mantenible, escalable y fácil de probar, permitiendo separar claramente las responsabilidades entre las distintas partes del sistema y facilitando su evolución futura.

2. Arquitectura General: Clean Architecture

La arquitectura del sistema sigue el enfoque de Clean Architecture, el cual propone organizar el software en capas donde el núcleo del sistema corresponde a la lógica del negocio, mientras que las capas externas contienen detalles de infraestructura o tecnologías específicas.

Capas principales:

- Capa de Presentación (API / UI)

Encargada de recibir las solicitudes de los usuarios o sistemas externos, procesar los datos de entrada y devolver las respuestas.

- Capa de Aplicación (Servicios o Casos de Uso)

Contiene la lógica de aplicación y coordina las operaciones del sistema.

- Capa de Dominio

Representa el núcleo del sistema y contiene las entidades del negocio, reglas y modelos principales.

- Capa de Infraestructura

Implementa los detalles técnicos como acceso a base de datos, integración con APIs externas y servicios de autenticación.

3. Arquitectura por Capas

La arquitectura por capas divide el sistema en componentes independientes donde cada capa cumple una función específica.

Capa de Presentación:

- Recibir solicitudes HTTP
- Validar datos de entrada
- Invocar servicios de la capa de aplicación
- Retornar respuestas al cliente

Capa de Aplicación:

- Implementar lógica de negocio
- Coordinar repositorios y servicios
- Aplicar reglas de negocio

Capa de Acceso a Datos:

- Consultar información
- Insertar registros
- Actualizar datos
- Eliminar registros
- Encapsular el uso del ORM o base de datos

4. Patrones y Estilos Arquitectónicos Implementados

Repository Pattern:

Se utiliza para abstraer el acceso a la base de datos mediante un repositorio genérico que encapsula operaciones CRUD.

Dependency Injection (Inyección de Dependencias):

Permite que los componentes del sistema reciban automáticamente las dependencias que necesitan, reduciendo el acoplamiento entre clases.

Options Pattern:

Permite mapear configuraciones del sistema a clases tipadas para facilitar su acceso y mantenimiento.

Middleware Pipeline:

Cadena de componentes que procesan cada solicitud HTTP antes de llegar al controlador, permitiendo agregar lógica como seguridad, validaciones y manejo de errores.

HttpClientFactory:

Permite gestionar correctamente los clientes HTTP utilizados para consumir servicios externos.

Singleton con Double Check Locking:

Garantiza que solo exista una instancia de ciertos componentes globales durante el ciclo de vida de la aplicación.

Autenticación y Autorización Centralizadas:
Implementadas mediante OpenID Connect para gestionar identidad y acceso de usuarios de forma segura.

Session State:
Permite almacenar temporalmente información del usuario durante su sesión, como roles, permisos y datos temporales.

5. Beneficios de la Arquitectura

Separación de responsabilidades:
Cada capa tiene funciones claras, reduciendo el acoplamiento entre componentes.

Facilidad de mantenimiento:
Los cambios en una capa no afectan directamente a otras partes del sistema.

Escalabilidad:
Permite agregar nuevas funcionalidades o integraciones de forma ordenada.

Mejor testabilidad:
Gracias a la inyección de dependencias se pueden realizar pruebas unitarias utilizando implementaciones simuladas.

Mayor seguridad:
La autenticación, autorización y manejo de solicitudes se controlan mediante middleware centralizados.

Trabajo colaborativo:
Permite que diferentes desarrolladores trabajen en distintas capas del sistema sin interferir entre sí.

6. Conclusión

La arquitectura propuesta combina principios de Clean Architecture, arquitectura por capas y diversos patrones de diseño con el objetivo de construir un sistema robusto, mantenible y escalable.

Esta estructura facilita la evolución del sistema a largo plazo, permitiendo adaptarse a nuevos requerimientos tecnológicos o de negocio sin comprometer la estabilidad del software.